

INTRODUCTION

The Campus Event Registration System is a desktop-based application created using Python Tkinter and MySQL. This project is designed to make the process of registering for college events easier and more organized.

In many colleges, event registrations are often done manually using paper forms or spreadsheets. This can lead to confusion, duplicate entries, and difficulty in managing student data. To solve this problem, this system provides a digital platform where students can register for events quickly through a graphical user interface.

The application allows students to: Enter their personal details, Select their department and year, Choose a club and event, View the event date automatically and Register for the event instantly.

All the entered details are stored safely in a MySQL database, making data management simple and efficient.

This project also helps in understanding how frontend GUI and backend database systems work together in a real application.

Objectives of the Project

The main objectives of this project are:

- To automate the college event registration process
- To reduce manual work and paperwork
- To create a simple and attractive user interface
- To store registration details securely in a database
- To improve accuracy and speed during registrations
- To learn Python GUI development and database connectivity

Tkinter Interface

Tkinter is the standard Graphical User Interface (GUI) library in Python. It is used to create desktop applications with windows, buttons, labels, text boxes, dropdown menus, and other interactive components.

Tkinter provides a simple way to design user-friendly interfaces without requiring advanced programming knowledge. It allows developers to build applications where users can interact visually instead of using only command-line inputs.

Tkinter works using an **event-driven programming approach**. This means the application responds to user actions such as button clicks, keyboard input, and dropdown selections. Each widget performs a specific function and can trigger events when interacted with.

Some commonly used Tkinter widgets in this project include:

Widget	Purpose
--------	---------

Label	Displays text such as headings, instructions, or messages on the GUI.
Entry	Accepts user input through a text field.
Button	Performs specific actions when clicked by the user.
Combobox	Creates dropdown menus for selecting options from a list.
Frame	Organizes and groups GUI components within the window layout.

Technology Used

Technology	Purpose
Python	Main programming language used for developing the application logic and functionalities.
Tkinter	Used for creating the Graphical User Interface (GUI) of the application.
MySQL	Used for storing and managing user and event-related data in the database.
mysql.connector	Connects the Python program with the MySQL database for executing queries.
ttk Module	Used for creating styled and advanced GUI widgets such as Combobox.

Command Used

Feature	Syntax	Working / Meaning
Main window	<code>root = Tk()</code>	Creates the main application window where all GUI elements are placed.
Window title	<code>root.title("Title")</code>	Sets the title displayed on the top bar of the application window.
Window size	<code>root.geometry("520x730")</code>	Defines the width and height of the application window.
Background color	<code>root.configure(bg="#0F111A")</code>	Sets the background color of the main window.
Label widget	<code>Label(root, text="...")</code>	Displays static text on the GUI (used for headings and field names).
Entry widget	<code>Entry(root)</code>	Creates an input field where the user can type data.
Button widget	<code>Button(root, command=function)</code>	Creates a clickable button that triggers a function when pressed.
Combobox widget	<code>ttk.Combobox(root)</code>	Creates a dropdown list allowing users to select one option from multiple

Feature	Syntax	Working / Meaning
		values.
Combobox values	combobox["values"] = (...)	Sets the list of selectable options inside the dropdown menu.
Get input value	.get()	Retrieves the current value entered or selected by the user in a widget.
Clear entry field	.delete(0, END)	Removes all text from an input field.
Set combobox value	.set("")	Clears or assigns a value to the dropdown selection.
Update label text	.config(text="...")	Dynamically changes the text displayed in a label.
Event binding	widget.bind("<<ComboboxSelected>>", func)	Calls a function automatically when a selection is made in the dropdown.
Nested dictionary	{club: {event: date}}	Stores structured event data for each club and its corresponding events.
MySQL connection	mysql.connector.connect()	Establishes connection between Python program and MySQL database.
Cursor object	db.cursor()	Used to execute SQL queries in the database.

Execute SQL query	<code>cursor.execute(query, values)</code>	Sends SQL command to database for execution.
Feature	Syntax	Working / Meaning
Commit changes	<code>db.commit()</code>	Permanently saves changes made in the database.
INSERT query	<code>INSERT INTO table VALUES (...)</code>	Inserts user data into database table.
Try-except block	<code>try: ... except:</code>	Handles errors like duplicate entries or database failures.
After delay function	<code>root.after(10000, func)</code>	Executes a function after a specified time delay (used for messages/animation).
Animation system	<code>after()</code> loop with index	Repeatedly updates label text to create moving animation effect.

Software Used:

- Python 3.x
- MySQL Server
- VS Code / PyCharm
- Windows Operating System

Code:

Part 1:

```
from tkinter import * from
tkinter import ttk import
mysql.connector

# DATABASE CONNECTION
db =
mysql.connector.connect(  hos
t="localhost",  user="root",
password="zorori",
    database="COLLEGEEVENT"
)

cursor = db.cursor()
```

Part 2:

```
events = {
    "Nova": {
        "ByteForge": "2026-06-10",
        "CodeStorm": "2026-06-12",
```

```

        "DesignRush": "2026-06-14"    },

    "Aura": {
        "Rhythm Night": "2026-06-18",
        "VerseWave": "2026-06-19",
        "StageSphere": "2026-06-20"
    }
}

# Update Event Dropdown def
update_events(event):
    selected_club = club_dropdown.get()
    event_dropdown["values"] = list( events[selected_club].keys() )
    event_dropdown.set("")
    date_label.config(text="Select Event")

# Update Event Date def
update_date(event):
    selected_club = club_dropdown.get()
    selected_event = event_dropdown.get() date
    = events[selected_club][selected_event]
    date_label.config(text=date)

```

Part 3:

```

def register():
    student_name = name_entry.get()

```



```

        usn = usn_entry.get()
        department = dept_dropdown.get()
        student_year = year_dropdown.get()
        club_name = club_dropdown.get()
        event_name = event_dropdown.get()
        event_date = date_label.cget("text")
        phone = phone_entry.get()

    try:

        query = """INSERT INTO REGISTRATION
(student_name, usn, department, student_year,          club_name,
event_name, event_date, phone)

VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s)"""

        values = (
            student_name, usn, department,
student_year, club_name,          event_name,
event_date, phone
        )

        cursor.execute(query, values)
        db.commit()

        status_label.config(
            text="Registration Successful!",
            fg="#B6FF00"
        )

```

```

        root.after(10000,
            lambda: status_label.config(text="")
        )

        # CLEAR INPUTS
        name_entry.delete(0, END)
        usn_entry.delete(0, END)
        phone_entry.delete(0, END)

        dept_dropdown.set("")
        year_dropdown.set("")          club_dropdown.set("")
        event_dropdown.set("")

        date_label.config(text="Select Event")

    except:
        status_label.config(
            text="Already Registered For This Event!",
            fg="#FF008A"
        )

        root.after(4000,
            lambda: status_label.config(text="")
        )

```

Part 4:

```
root = Tk()
root.title("CampusHub Event Registration")
root.geometry("520x730") root.configure(bg="#0F111A")
```

```
style = ttk.Style() style.theme_use("clam")
```

```
# REGISTER BUTTON
```

```
register_button = Button(
    root,
    text="REGISTER",
    command=register,
    bg="#00E5FF", fg="#0F111A",
    font=("Helvetica", 12, "bold"),
    cursor="hand2"
)
```

```
# ANIMATION SYSTEM animation_texts
```

```
= [
    "SYSTEM ACTIVE ■",
    "SYSTEM ACTIVE ■■",
    "SYSTEM ACTIVE ■■■",
    "SYSTEM ACTIVE ■■■■",
    "SYSTEM ACTIVE ■■■■■"
]
```

```
animation_index = 0
```

```
animation_label = Label(root, text="", bg="#0F111A", fg="#00E5FF")
```

```
def animate():
```

```
    global animation_index
```

```
    animation_label.config(text=animation_texts[animation_index])
```

```
    animation_index = (animation_index + 1) % len(animation_texts)
```

```
    root.after(300, animate)
```

```
animate() root.mainloop()
```

CampusHub Event Registration

Student Name

USN

Department

Academic Year

Club

Event

Event Date

Select Event

Phone Number

Registration Successful!

REGISTER

SHOW REGISTRATIONS

SYSTEM ACTIVE

CampusHub Event Registration

Student Name

shreedha

USN

4mc24cs175

Department

CSE

Academic Year

2

Club

Pulse

Event

Green Pjise

Event Date

2026-06-23

Phone Number

9837234432

REGISTER

SHOW REGISTRATIONS

SYSTEM ACTIVE

```
Run event
/Users/shreedha.m/Desktop/dbms tk/.venv1/bin/python /Users/shreedha.m/Desktop/dbms tk/event.py

REGISTERED STUDENTS

(23, 'shreedha.m.m', '4mc24cs175', 'CSE', 2, 'Aura', 'VerseWave', datetime.date(2026, 6, 19), '9876543210')
(24, 'shravya jain', '4mc24cs173', 'ISE', 3, 'Pulse', 'EcoNova', datetime.date(2026, 6, 22), '8465738294')
(25, 'tsering yangdol', '4mc23cs202', 'AIML', 4, 'Aura', 'StageSphere', datetime.date(2026, 6, 20), '8739208564')
```

```
mysql> select* from REGISTRATION;
```

reg_id	student_name	usn	department	student_year	club_name	event_name	event_date	phone
23	shreedha.m.m	4mc24cs175	CSE	2	Aura	VerseWave	2026-06-19	9876543210
24	shravya jain	4mc24cs173	ISE	3	Pulse	EcoNova	2026-06-22	8465738294
25	tsering yangdol	4mc23cs202	AIML	4	Aura	StageSphere	2026-06-20	8739208564

2 rows in set (0.002 sec)

Advantages of the System

- Easy and simple to use
- Reduces manual registration work

- Faster and more accurate process
- Stores data securely
- Improves event management
- Attractive graphical interface

Limitations of the System

- Works only on systems with MySQL installed
- Cannot be accessed online
- No admin login system
- Limited to desktop use only

Future Enhancements

- Add admin dashboard
- Add login authentication
- Send confirmation emails
- Generate registration receipts
- Convert into web or mobile application
- Add event seat limit system

Conclusion

The Campus Event Registration System successfully automates the process of registering students for college events using Python Tkinter and MySQL.

The project combines GUI design, database connectivity, and event handling into a single application. It provides a smooth and

userfriendly registration experience while storing data efficiently in a structured database.

Through this project, important concepts such as Python programming, Tkinter GUI development, MySQL database operations, and full application integration were understood and implemented successfully.